

Architettura degli Elaboratori

Vettori, matrici e pseudoistruzioni



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini,
Iacopo Masi, Claudio di Ciccio

[S&PdC] Capitolo 2



Modifica - Iterazioni (ciclo for) con mezze parole

Esempio C

```
int N = 10;

for (i=0 ; i<N ; i++)
{
    // codice da ripetere
}

// codice seguente
```

Esempio assembly

```
.data
N: .half 10

.text
# uso il registro t0 per l'indice i
# uso il registro t1 per il limite N
xor t0, t0, t0          # azzero i
lhu t1, N                # limite del ciclo
cicloFor:
    bge t0, t1, endFor   # test i >= N
    # codice da ripetere
    addi t0, t0, 1        # i += 1
    j cicloFor            # jump back
endFor:
    # codice seguente
```

Vettori e matrici

Vedremo:

- Vettori: manipolazione con indici e con puntatori
- Matrici a 2, 3 ed N dimensioni
- Esempi di programmi

Vettore:

sequenza di **N elementi** di **dimensioni uguali**

consecutivi in memoria

indirizzabili per indice (da 0 a N-1)

dimensione totale = $N \times \text{dimensione_elemento}$

Si possono definire staticamente nella sezione **.data** del programma assembly
usando un'**etichetta** per indicare l'**indirizzo del primo elemento** del vettore

Per indirizzare l'**elemento i-esimo** bisogna aggiungere l'offset
 $i \times \text{dimensione_elemento}$

Vettore di word vs half word

vettore di word a partire da 0x00001004

vettore di half word a partire da 0x0001000

Vettore di word vs half word

vettore di word a partire da 0x00001004

vect: .word 3,-1

0x00001004 -> 0x3

0x00001008 -> 0xffffffff

32			0
00	00	00	03
ff	ff	ff	ff

vettore di half word a partire da 0x0001000

vect: .half 3,-1

0x00001000 -> 0x3

0x00001002 -> 0xffff

32			0
ff	ff	00	03

Vettore di word vs half word

vettore di word a partire da 0x00001004

```
vect: .word 3,-1
```

0x00001004 -> 0x3

0x00001008 -> 0xffffffff

vettore di half word a partire da 0x00001000

```
vect: .half 3,-1
```

0x00001000 -> 0x3

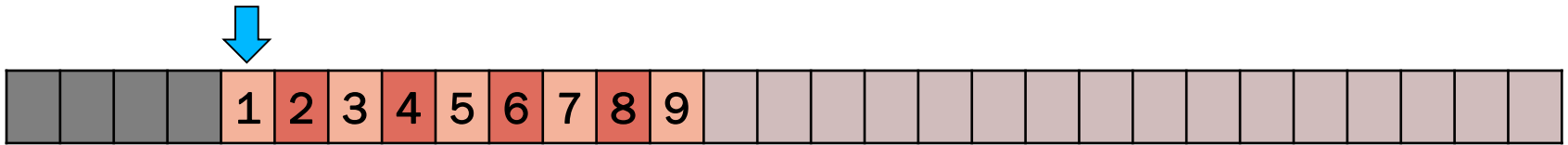
0x00001002 -> 0xffff

FF	0x0000100B
FF	0x0000100A
FF	0x00001009
FF	0x00001008
00	0x00001007
00	0x00001006
00	0x00001005
03	0x00001004
FF	0x00001003
FF	0x00001002
00	0x00001001
03	0x00001000

Vettori di byte in memoria

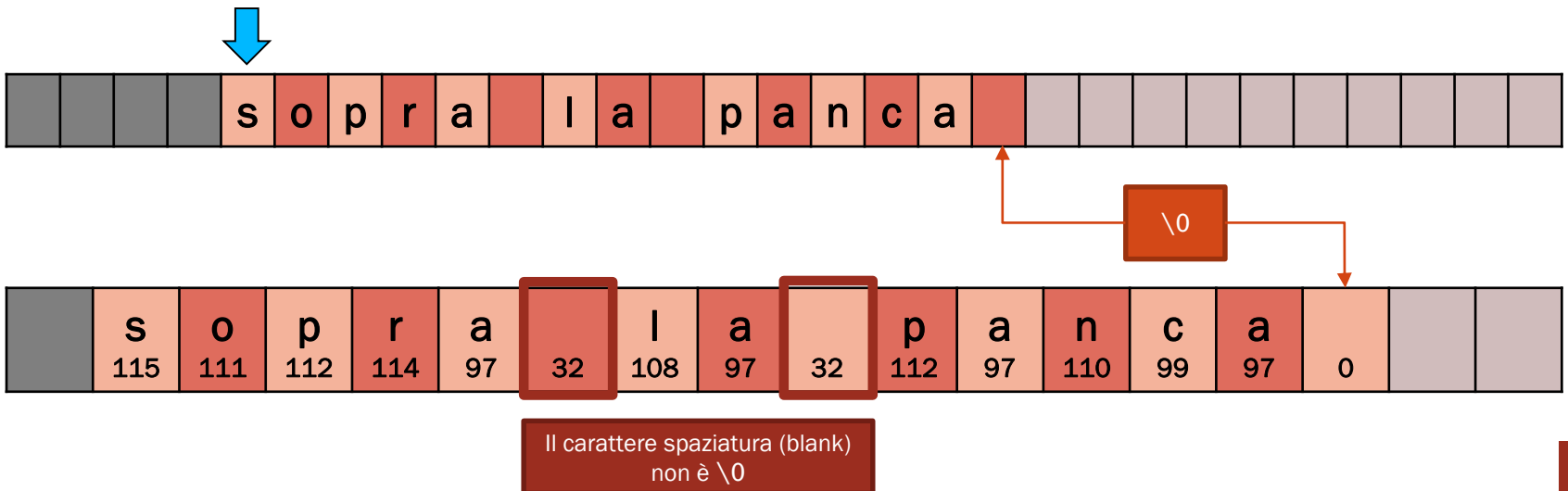
Vettore di **byte** (valori interi da 0 a 255)

label1: .byte 1, 2, 3, 4, 5, 6, 7, 8, 9



Testo: vettore di caratteri (**byte**) seguiti da `\0` (carattere codificato con zero, 0x0)

label2: .ascii "sopra la panca"

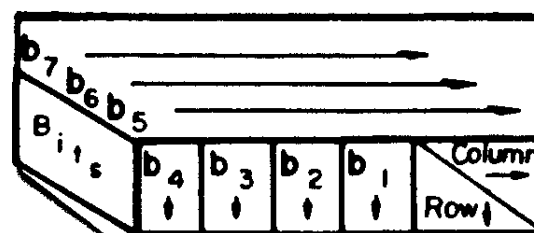


American Standard Code for Information Interchange

<https://theasciicode.com.ar>

USASCII code chart

A → 0x41 = 0100 0001
a → 0x61 = 0110 0001

					0 0 0		0 0 1		0 1 0		0 1 1		1 0 0		1 0 1		1 1 0		1 1 1	
b ₄ ↑	b ₃ ↑	b ₂ ↑	b ₁ ↑	Column Row ↘	0	1	2	3	4	5	6	7								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p								
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q								
0	0	1	0	2	STX	DC2	"	2	B	R	b	r								
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s								
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t								
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u								
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v								
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w								
1	0	0	0	8	BS	CAN	(	8	H	X	h	x								
1	0	0	1	9	HT	EM	)	9	I	Y	i	y								
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z								
1	0	1	1	11	VT	ESC	+	;	K	[	k	{								
1	1	0	0	12	FF	FS	,	<	L	\	l									
1	1	0	1	13	CR	GS	-	=	M	]	m	}								
1	1	1	0	14	SO	RS	.	>	N	^	n	~								
1	1	1	1	15	SI	US	/	?	O	_	o	DEL								

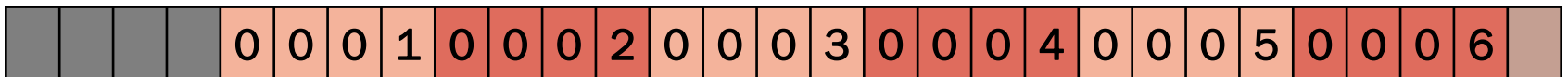
Endianess: chiarimenti

Il concetto di **Endianess** (se un dato sistema è little endian o big endian) ha a che fare con:

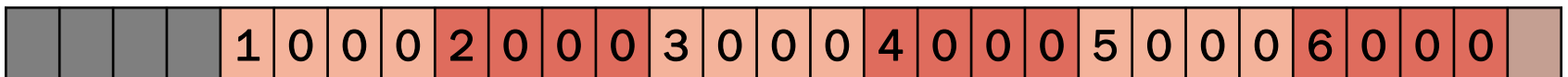
- indirizzamento in memoria (che avviene **per byte**) – ogni indirizzo specifica **il singolo byte**
- come i **byte** di una **word** sono ordinati in memoria

L'architettura RISC-V permette l'ordinamento dei byte di una word in due modi:

- **Big-endian** (o network-order, usato da Java e dalle CPU SPARC Sun/Oracle o embedded)
i byte della word sono memorizzati dal **most** significant *byte* al **least** significant *byte*



- **Little-endian** (usato dalle CPU Intel, ad es. Windows, Linux e da RARS)
i byte della word sono memorizzati dal **least** significant *byte* al **most** significant *byte*



Endianess: chiarimenti

Il concetto di **Endianess** (se un dato sistema è little endian o big endian) ha a che fare con:

- indirizzamento in memoria (che avviene **per byte**) – ogni indirizzo specifica il **singolo byte**
- come i **byte** di una **word** sono ordinati in memoria

L'architettura RISC-V permette l'ordinamento dei byte di una word in:

- **Little-endian** (usato dalle CPU Intel, ad es. Windows, Linux e da **RARS**)

i byte della word sono memorizzati dal **least** significant *byte* al **most** significant *byte*

				1	0	0	0	2	0	0	0	3	0	0	0	4	0	0	0	5	0	0	0	6	0	0	0	
				0	0	0	0	1	1	1	1	2	2	2	2	3	3	3	3	4	4	4	4	5	5	5	5	
				0	4	8	C	0	4	8	C	0	4	8	C	0	4	8	C	0	4	8	C	0	4	8	C	

Esempio – vettore di word con 0x01020304, 0x4321

.text

Big Endian

Little Endian

Esempio – vettore di word con 0x01020304, 0x4321

.text

word1: .word 0x01020304

word2: .word 0x4321

Big Endian

01 02 03 04

00 00 43 21

Little Endian

04 03 02 01

21 43 00 00

E i byte?

Provate

vector: .byte 1,2,3,4 (a quale word equivale?)

RARS e i vettori

RARS è little-endian

- nella finestra dei dati le **stringhe** sono visualizzate come **gruppi di 4 caratteri rovesciati**

The screenshot shows the RARS interface with the 'Settings' menu open. The 'Show Labels Window (symbol table)' checkbox is checked. A red arrow points from this checkbox to the 'Labels' window. The 'Labels' window shows a list of labels: 'label1' at address 0x10010000, 'label2' at 0x10010002, and 'label3' at 0x10010018. A red box highlights the 'Labels' window. Another red arrow points from the 'Labels' window to the 'Data Segment' window. The 'Data Segment' window shows a table of memory addresses and their values. The first row, starting at address 0x10010000, contains the characters 'r p o s a l a n a p . \0 a c'. A red box highlights this row. At the bottom of the 'Data Segment' window, the 'ASCII' checkbox is checked, and a red box highlights it.

label1: .asciiz "sopra la panca"

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	r	p	o	s	a	l	a	n
0x10010020	.	\0	a	c	.	.	.	.
0x10010040	\0	\0	\0	\0	\0	\0	\0	\0
0x10010060	\0	\0	\0	\0	\0	\0	\0	\0
0x10010080	\0	\0	\0	\0	\0	\0	\0	\0
0x100100a0	\0	\0	\0	\0	\0	\0	\0	\0
0x100100c0	\0	\0	\0	\0	\0	\0	\0	\0
0x100100e0	\0	\0	\0	\0	\0	\0	\0	\0

Accesso agli elementi per indice

Indirizzo dell'elemento i = indirizzo base del vettore + $i \times \text{dimensione_elemento}$

Esempio: (frammento che calcola l'indirizzo di un elemento in un vettore di word)

t0 contiene l'indice dell'elemento (e.g., 2)

t1 contiene l'indirizzo del vettore (e.g., 0x10010040)

in t2 si ottiene l'indirizzo dell'elemento (da caricare da, o scrivere in, memoria)

`slli t2, t0, 2` # word $\rightarrow$ 4 byte; shift di due bit $\rightarrow$ moltiplicazione per 4

`add t2, t2, t1` # indirizzo del vettore + spiazzamento

`lw s0, (t2)` # carica word

Per elementi di dimensioni diverse (half word o byte o altro) va cambiata la prima istruzione.

Se il vettore è allocato staticamente (i.e., viene dichiarato in sezione `.data`), bisogna caricare l'indirizzo base con

`la t1, Array`

Esempio

Esempio: (frammento che calcola l'indirizzo di un elemento in un vettore di word)

t0 contiene l'indice dell'elemento (e.g., 2)

t1 contiene l'indirizzo del vettore (e.g., 0x10010040)

in t2 si ottiene l'indirizzo dell'elemento (da caricare da, o scrivere in, memoria)

slli t2, t0, 2 # word → 4 byte; shift di due bit → moltiplicazione per 4

add t2, t2, t1 # i*t0+t1

Cicli

Nei cicli si possono usare due stili di scansione di un vettore

Scansione per indice

- Pro:
 - comoda se si deve usare l'indice dell'elemento per controlli o altro
 - l'incremento dell'indice non dipende dalla dimensione degli elementi
 - comoda se il vettore è allocato staticamente (nella sezione `.data`)
- Contro:
 - bisogna convertire ogni volta l'indice nel corrispondente offset in byte

Scansione per puntatore (ovvero manipolando direttamente indirizzi in memoria)

- Pro:
 - si lavora direttamente su indirizzi in memoria
 - ci sono meno calcoli nel ciclo
- Contro:
 - non si ha a disposizione l'indice dell'elemento
 - l'incremento del puntatore dipende dalla dimensione degli elementi
 - bisogna calcolare l'indirizzo successivo all'ultimo elemento

Esempio (con indice)

Esempio: somma degli elementi di un vettore di word a **posizione** divisibile per tre

.data

```
Vettore: .word 1, 2, 3, 4, 5, 6, 7, 8, 9 # vettore da sommare
N:       .word 9                        # numero di elementi
Somma:   .word 0                        # risultato
```

.text

```
main:    li      t0, 0                    # i = 0
         lw      t1, N                    # lettura di N
         li      t2, 0                    # somma = 0
loop:    bge     t0, t1, fine              # è finito il ciclo?
         slli    t3, t0, 2                # offset: i*4
         la      t4, Vettore              # indirizzo di memoria
         add     t3, t3, t4                # più spiazzamento
         lw      t3, (t3)                  # lettura di Vettore[i] (riuso t3)
         add     t2, t2, t3                # somma += Vettore[i]
         addi    t0, t0, 3                 # i += 3
         j       loop                     # riparte il ciclo
fine:    la      t0, Somma                 # carica l'indirizzo di Somma
         sw      t2, (t0)                  # memorizzo il risultato
```

Esempio (con puntatori)

.data

```
Vettore: .word 1, 2, 3, 4, 5, 6, 7, 8, 9 # vettore da sommare
N:       .word 9                        # numero di elementi
Somma:   .word 0                        # risultato
```

.text

```
main:    lw      t1, N                  # lettura di N
         la      t0, Vettore            # indirizzo di Vettore
         slli    t1, t1, 2              # dimensione = N * 4
         add     t1, t1, t0             # fine = ind.Vettore + dim
         li      t2, 0                  # somma = 0
loop:    bge     t0, t1, fine            # è finito il ciclo?
         lw      t3, (t0)               # lettura di Vettore[i]
         add     t2, t2, t3             # somma += Vettore[i]
         addi    t0, t0, 12              # i += 3 * dim_elemento
         j       loop
fine:    la      t0, Somma               # carica l'indirizzo di Somma
         sw      t2, (t0)               # memorizzo il risultato
```

Esempio (con puntatori)

.data

```
Vettore: .word 1, 2, 3, 4, 5, 6, 7, 8, 9 # vettore da sommare
N:       .word 9                        # numero di elementi
Somma:   .word 0                        # risultato
```

.text

```
main:    lw      t1, N                  # lettura di N
         la      t0, Vettore            # indirizzo di Vettore
         slli    t1, t1, 2              # dimensione = N * 4
         add     t1, t1, t0             # fine = ind.Vettore + dim
         li      t2, 0                  # somma = 0
loop:    bge     t0, t1, fine            # è finito il ciclo?
         lw      t3, (t0)                # lettura di Vettore[i]
         add     t2, t2, t3              # somma += Vettore[i]
         addi    t0, t0, 12              # i += 3 * dim_elemento
         j       loop
fine:    sw      t2, 4(t1)                # memorizzo il risultato (t1 è
                                         l'indirizzo di N)
```

Matrici: vettori di vettori

Una matrice $M \times N$ è altro una successione di M vettori, ciascuno di N elementi

- il numero di elementi totali è: $M \times N$
- la dimensione totale in byte è: $M \times N \times \text{dimensione_elemento}$
- la si definisce staticamente come un vettore contenente $M \times N$ elementi uguali

Matrice: **.word 0:91** # spazio per una matrice 7×13 word

L'elemento **e**,
di coordinate
x=9, y=2 si trova
ad una distanza di:
• 2 righe
• più 9 elementi
dall'inizio, ovvero ad
un offset di
 $2 \times 13 + 9 = 35$ word
cioè
 $35 \times 4 = 140$ byte

13 colonne

7 righe	0	1	2	3	4	5	6	7	8	9	10	11	12	0
	13	14	15	16	17	18	19	20	21	22	23	24	25	1
	26	27	28	29	30	31	32	33	34	e				2
														3
														4
														5
														6
	0	1	2	3	4	5	6	7	8	9	10	11	12	

x

y

Matrici: ordinamento in memoria

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

0	1	2	3	4	5
6	7	8	9	10	11
12	13	14	15	16	17
18	19	20	21	22	23
24	25	26	27	28	29
30	31	32	33	34	35

C- Style

Fortran- Style

0
6
12
18
24
30
1
7
13
19
25
31
2
8
14
20
26
32

Matrice 3d

2 righe, 3 colonne, 2 strati

0	1	2		
3	4	5	8	
	9	10	11	14
		15	16	17

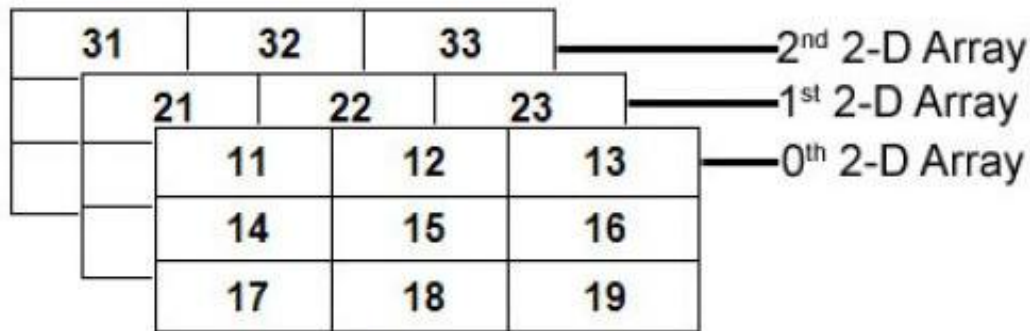


0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

Matrici 2D e 3D

Questo è esattamente il calcolo svolto dal compilatore C, per esempio

scrivere	<code>matrice[y][x]</code>
per una matrice di interi definita	<code>int matrice[NUM_RIGHE][NUM_COL]</code>
equivale a usare l'indirizzo	<code>matrice + (y*NUM_COL+x)*sizeof(int)</code>



Matrici a 3 dimensioni

Una matrice 3D di **dimensioni** $M \times N \times P$ è una successione di P matrici 2D grandi $M \times N$

L'elemento a coordinate x, y, z è preceduto da:

z «strati» (matrici $M \times N$ formate da $M \cdot N$ elementi)

y «righe» di M elementi sullo stesso strato

x «elementi» sulla stessa riga e strato

Quindi l'elemento si trova a $z * (M * N) + y * N + x$ elementi dall'inizio della matrice 3D

e la sua posizione in memoria è `indirizzo_matrice + (z * (M * N) + y * N + x) * dim_el.`

Riprendiamo dall'esercizio



SAPIENZA
UNIVERSITÀ DI ROMA

Compito per casa



1. Installare **RARS** e prendere familiarità con interfaccia
2. Scrivere i programmi visti a lezione e provare ad eseguirli e debuggarli passo passo
 - Controllare come cambiano i registri sulla destra in base ai passi svolti.
 - Ispezionare come cambiano le pseudo-istruzioni immesse nelle istruzioni che poi svolge veramente il calcolatore
3. Partendo dal programma che trova il max in un vettore scrivere un programma in linguaggio assembly RISC-V con RARS che dato un vettore ingresso **vector** e la sua dimensione **N** calcoli due somme dei numeri del vettore.
 1. La prima somma deve sommare i valori del vettore di indice **dispari**. (Indice parte da 1)
 2. La seconda somma deve sommare i valori di un vettore con **indice pari**. (Indice parte da 0)



Registro	Nome
x0	zero
x1	ra
x2	sp
x3	gp
x4	tp
x5-x7	t0-t2
x8	s0 / fp
x9	s1
x10-x11	a0-a1
x12-x17	a2-a7
x18-x27	s2-s11
x28-x31	t3-t6

Vector .word 4 -1 5 500 0 10000 -256

N .word 5

Somme: .word 0, 0

Una volta scritto con questi dati cambiate **N** e controllate se continua a funzionare

Una possibile soluzione

```
.data
#####
# Declaration of constants and variables in memory
#####
Array: .word 4, -1, 5, 500, 0, 10000, -256
N:      .word 7
#Array: .word 1, 2, 2, 4
#N:      .word 4
Sums:    .word 0, 0

.text
#####
# Initialisation and loading
#####
and t1, t1, zero # $t1: cursor Index for the Array
add t2, zero, x0 # $t2: temporary value loaded from Array[$t1]
addi t3, zero, 0 # $t3: a flag for 3ven elements (1 if even), or I could look at the parity of a counter
li t5, 0 # $t5: off5et
andi s3, s3, 0 # $s3: sum of 3ven elements in Array
xor s0, s0, s0 # $s0: sum of 0dd elements in Array

lw s7, N # $s7: the length of Array ($s7 ← N)

#####
# Core business
#####
while:
    bge t1, s7, whileEnd # Exit the cycle if the cursor goes beyond the length of Array
# {
    or t3, t3, x0 # this plus next: nor
    not t3, t3 # Switch the flag. When $t3 == 0, $t3 ← 1. When $t3 == 1, $t3 ← 0
    slli t5, t1, 2 # offset = index × 4

    la s10, Array
    add s10, s10, t5
    lw t2, (s10) # load array value at Base + offset (indexed absolute)
    if:
        bnez t3, else # Is $t3 != 0 ? (even)
    then:
        # If not, $t2 contains a value at an odd index
        add s0, s0, t2 # Add the content of Array[$t1] to $s0 (0dd)
        j endif # Exit the if-then-else block
    else:
        # If $t2 contains a value at an even index
        add s3, s3, t2 # Add the content of Array[$t5] to $s3 (3ven)
    endif:
        addi t1, t1, 1 # Increase the index

    j while
# }
whileEnd:
# Store the results in Sums
la s10, Sums
sw s3, (s10) # Store the sum of 3even elements is Sums[0]
sw s0, 4(s10) # Store the sum of 0dd elements is Sums[1] (again, remember we are considering 4-byte words)
```